คำตอบแบบทดสอบ Programmer Assessment Test

ผู้จัดทำ: ซีรีน ยังปากน้ำ

GitHub คำตอบและโค้ด: https://github.com/srseen/southland_test

ข้อ 1 — การตรวจสอบปัญหาเว็บไซต์เบื้องต้น

โจทย์: หากพบว่าโปรแกรมและเว็บไซต์ที่พัฒนาขึ้นใช้งานแล้วมีปัญหาใช้งานไม่ได้ เราจะตรวจสอบปัญหาเบื้องต้นยังไง

หลักการ: ระบุและจำกัดขอบเขตของข้อผิดพลาดที่เกิดขึ้นเพื่อค้นหาสาเหตุที่แท้จริงของปัญหา

ขั้นตอนการตรวจสอบ:

1. **จำลองสถานการณ์ที่เกิดปัญหา (Reproduce the Problem):**
    - ทดลองใช้งานเว็บไซต์เพื่อระบุจุดที่เกิดปัญหาและขั้นตอนที่นำไปสู่ปัญหานั้น
    - ตรวจสอบว่าปัญหานี้เกิดขึ้นกับผู้ใช้งานทุกคนหรือไม่ และเกิดขึ้นกับเบราว์เซอร์/อุปกรณ์ทุกประเภทหรือไม่
    - หากไม่สามารถจำลองสถานการณ์ได้ อาจจำเป็นต้องสอบถามข้อมูลเพิ่มเติมจากผู้ใช้งาน
2. **พิจารณาสภาพแวดล้อมที่เกิดปัญหา (Development หรือ Production):**
    - การแก้ไขปัญหาจะแตกต่างกันไปขึ้นอยู่กับสภาพแวดล้อมที่เกิดปัญหา
3. **ตรวจสอบข้อผิดพลาดผ่าน Developer Tools (2 จุดหลัก):**
    - **Console:** หากเป็นข้อผิดพลาด JavaScript แสดงว่าเป็นปัญหาที่ฝั่ง Frontend
    - **Network:** ตรวจสอบ HTTP Status Code เช่น:
        - `400 Bad Request` : คำขอไม่ถูกต้อง
        - `401 Unauthorized` / `403 Forbidden` : ปัญหาการอนุญาตหรือสิทธิ์การเข้าถึง

        - `404 Not Found` : ไม่พบทรัพยากรที่ร้องขอ
        - `500 Internal Server Error` : ข้อผิดพลาดที่ฝั่งเซิร์ฟเวอร์

4. **ตรวจสอบบันทึกข้อผิดพลาด (Server Logs):**
    - ค้นหาข้อผิดพลาดที่เกี่ยวข้องกับการเชื่อมต่อฐานข้อมูล (DB Connect Error), การตั้งค่าที่ไม่ถูกต้อง (Config Error) หรือข้อผิดพลาดจากบริการภายนอก (Third-Party Error)

ข้อ 2 — Loop Trace: หาค่า i

โจทย์: จากส่วนของโปรแกรมที่กำหนดให้ หลังจากจบการทำงานแล้ว ค่าของ `i` มีค่าเป็นเท่าใด

Pseudocode:

```
 x := 1;
i := 1;
while (x <= 1000)
begin
  x := 2^x;
  i := i + 1;
end;
```

Trace การทำงาน:

รอบที่	x (ก่อน)	x := 2^x	i (หลัง)
1	1	2^1 = 2	2
2	2	2^2 = 4	3
3	4	2^4 = 16	4
4	16	2^16 = 65,536	5

เมื่อ `x` มีค่าเป็น 65,536 ซึ่งมากกว่า 1000 ทำให้เงื่อนไข `x <= 1000` เป็นเท็จ การทำงานของลูปจึงหยุดลง

คำตอบ: `i = 5`

ข้อ 3 — Nested Loop: หาค่า r (n=256)

โจทย์: ถ้า `n` มีค่าเท่ากับ 256 หลังจากจบการทำงานแล้ว `r` มีค่าเป็นเท่าใด

Pseudocode:

```
 i := n;
r := 0;
while (i >= 1) do
begin
  for j := 1 to n do
    r := r + 1;
  i := i / 2;
end;
```

Trace การทำงาน:

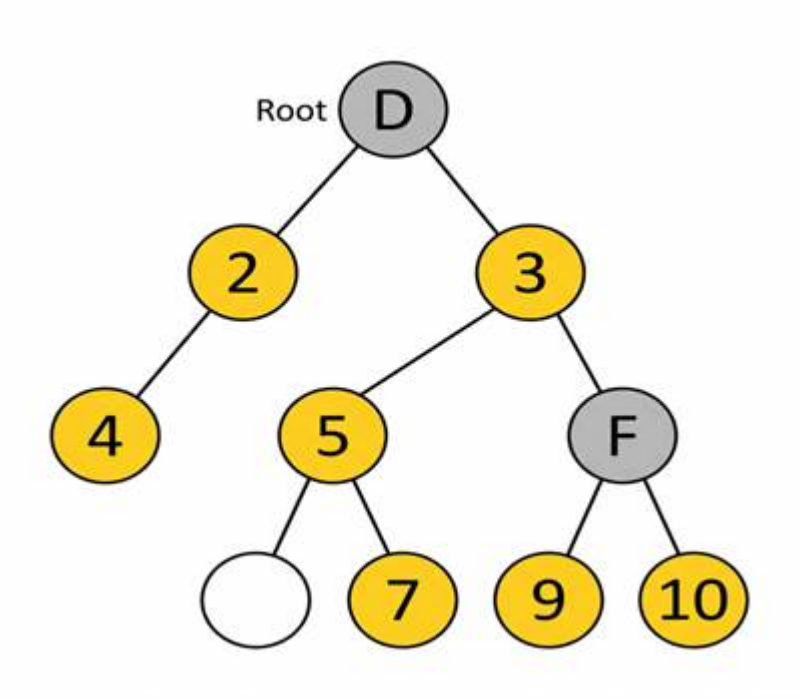
- **Outer loop:** ค่า `i` จะถูกหารด้วย 2 ในทุกรอบการทำงาน ซึ่งจะมีการทำงานทั้งหมด 9 รอบ (256, 128, 64, 32, 16, 8, 4, 2, 1)
- **Inner loop:** ในแต่ละรอบของ Outer loop, Inner loop จะทำงาน `n` ครั้ง (256 ครั้ง) เสมอ

รอบที่	i (ก่อน)	i (หลัง)	จำนวนครั้งของ Inner loop	ค่า r (สะสม)
1	256	128	256	256
2	128	64	256	512
3	64	32	256	768
4	32	16	256	1,024
5	16	8	256	1,280
6	8	4	256	1,536
7	4	2	256	1,792
8	2	1	256	2,048
9	1	0	256	2,304

สูตร: `r = จำนวนรอบของ Outer loop × n = 9 × 256 = 2,304`

คำตอบ: `r = 2,304`

ข้อ 4 — Tree Traversal: หาผลรวมตัวเลข



โจทย์: ให้เขียนภาษาโปรแกรมอะไรก็ได้หรือ Pseudocode หาผลรวมของตัวเลขใน Tree

แนวคิด:

- ใช้ Depth First Search (DFS) เพื่อวน traverse ทุก node ใน Tree
- หาก node เป็นตัวเลข ให้รวมค่าเข้ากับผลรวม
- หาก node เป็นตัวอักษร (เช่น D, F) หรือเป็นค่าว่าง ให้ข้ามไป

Pseudocode:

```
function sumNumbers(node):
  if node is null then return 0

  sum = 0
  if node.value is number then
    sum = node.value

  for each child in node.children:
    sum = sum + sumNumbers(child)

  return sum
```

ตัวเลขที่นับ: `2 + 3 + 4 + 5 + 7 + 9 + 10`

คำตอบ: ผลรวม = 40

ข้อ 5 — Database Design & Normalization

โจทย์: Design และ Normalization Database Table ข้อมูลลูกค้า

Fields: รหัส, คำนำหน้า, ชื่อ, สกุล, ที่อยู่, บัตร ปชช, เพศ, วันเกิด, ขนาดลูกค้า (S, M, B), เกรดลูกค้า (A, B, C, D)

เงื่อนไข: ลูกค้าบางรายอาจจะเป็นลูกค้าย่อยของลูกค้าหลักได้

Normalization เป็น 4 ตาราง:

ตาราง	หน้าที่
`prefixes`	เก็บคำนำหน้าชื่อ (นาย, นาง, นางสาว)
`customer_sizes`	เก็บขนาดลูกค้า (S, M, B)
`customer_grades`	เก็บเกรดลูกค้า (A, B, C, D)
`customers`	ตารางหลัก + Self-Referencing Foreign Key

จุดสำคัญ: `parent_customer_id` เป็น Self-Referencing Foreign Key ชี้กลับไปที่ตาราง `customers` เอง

- ถ้า `NULL` = ลูกค้าหลัก

- ถ้ามีค่า = ลูกค้าย่อย

SQL สร้างตาราง:

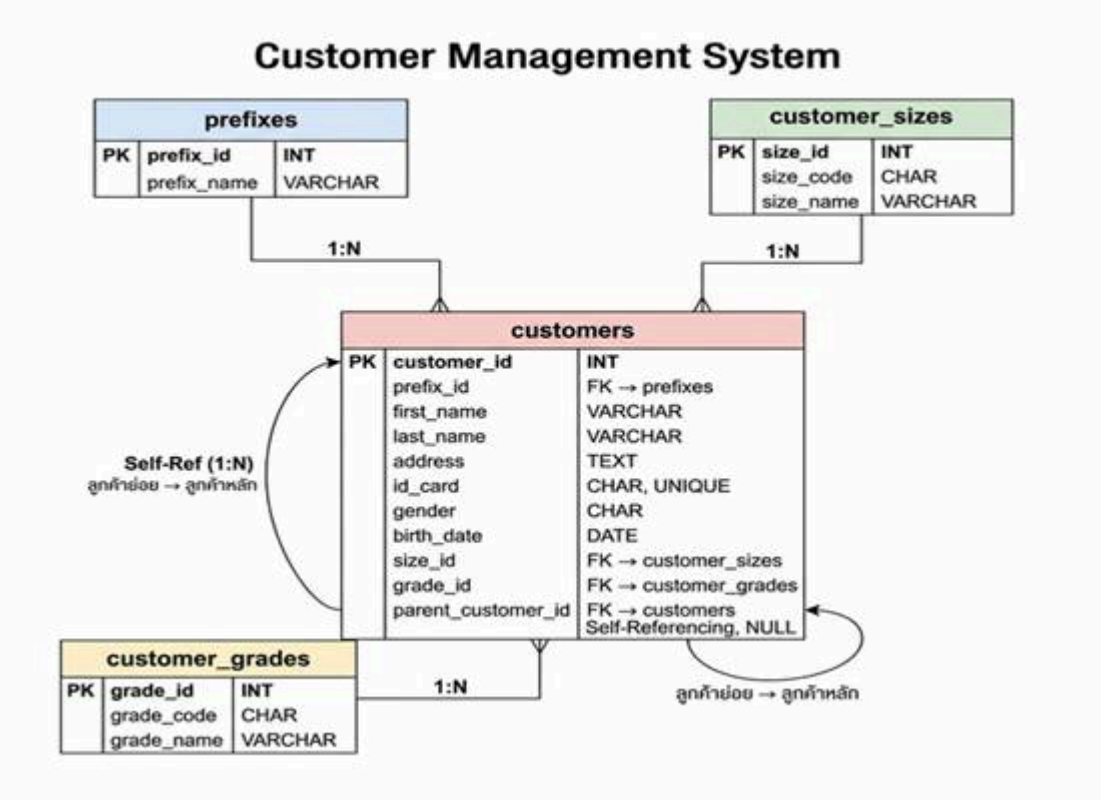
```sql
CREATE TABLE prefixes (
    prefix_id INT PRIMARY KEY AUTO_INCREMENT,
    prefix_name VARCHAR(20) NOT NULL
);

CREATE TABLE customer_sizes (
    size_id INT PRIMARY KEY AUTO_INCREMENT,
    size_code CHAR(1) NOT NULL UNIQUE,
    size_name VARCHAR(50) NOT NULL
);

CREATE TABLE customer_grades (
    grade_id INT PRIMARY KEY AUTO_INCREMENT,
    grade_code CHAR(1) NOT NULL UNIQUE,
    grade_name VARCHAR(50) NOT NULL
);

CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    prefix_id INT NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    address TEXT,
    id_card CHAR(13) NOT NULL UNIQUE,
    gender CHAR(1) NOT NULL,
    birth_date DATE,
    size_id INT NOT NULL,
    grade_id INT NOT NULL,
    parent_customer_id INT NULL,
    FOREIGN KEY (prefix_id) REFERENCES prefixes(prefix_id),
    FOREIGN KEY (size_id) REFERENCES customer_sizes(size_id),
    FOREIGN KEY (grade_id) REFERENCES customer_grades(grade_id),
    FOREIGN KEY (parent_customer_id) REFERENCES customers(customer_id)
);
```

ER Diagram:



ข้อ 6 — SQL Query ค้นหาลูกค้าย่อย

โจทย์: จากข้อที่ 5 ให้ทำการค้นหาข้อมูลลูกค้าย่อยเท่านั้นและให้แสดงว่าเป็นลูกค้าย่อยของลูกค้าหลักรายไหน

แนวคิด:

- ลูกค้าย่อยคือลูกค้าที่มี `parent_customer_id IS NOT NULL`
- ใช้ Self JOIN เพื่อเชื่อมตาราง `customers` กับตัวเอง เพื่อดึงข้อมูลลูกค้าหลักมาแสดง

SQL Query:

```sql
SELECT
    sub.customer_id AS sub_customer_id,
    p1.prefix_name AS sub_prefix,
    sub.first_name AS sub_first_name,
    sub.last_name AS sub_last_name,
    main.customer_id AS main_customer_id,
    p2.prefix_name AS main_prefix,
    main.first_name AS main_first_name,
    main.last_name AS main_last_name
FROM customers sub
INNER JOIN customers main ON sub.parent_customer_id = main.customer_id
INNER JOIN prefixes p1 ON sub.prefix_id = p1.prefix_id
INNER JOIN prefixes p2 ON main.prefix_id = p2.prefix_id;
```

คำอธิบาย Query:

ส่วนของ Query	คำอธิบาย
`FROM customers sub`	เริ่มจากตาราง `customers` โดยตั้งชื่อแทนว่า `sub` (ลูกค้าย่อย)
`INNER JOIN customers main ON sub.parent_customer_id = main.customer_id`	เชื่อม (join) ตาราง `customers` กับตัวเอง โดยตั้งชื่อแทนว่า `main` (ลูกค้าหลัก) และเชื่อมด้วย `parent_customer_id` ของลูกค้าย่อยกับ `customer_id` ของลูกค้าหลัก
`INNER JOIN prefixes p1 ON sub.prefix_id = p1.prefix_id`	ดึงคำนำหน้าของลูกค้าย่อยจากตาราง `prefixes`
`INNER JOIN prefixes p2 ON main.prefix_id = p2.prefix_id`	ดึงคำนำหน้าของลูกค้าหลักจากตาราง `prefixes`

หมายเหตุ: การใช้ `INNER JOIN` จะทำให้ได้เฉพาะลูกค้าที่มี `parent_customer_id` เท่านั้น (ลูกค้าย่อย) ลูกค้าหลักที่ `parent_customer_id IS NULL` จะไม่ถูกดึงมาแสดงในผลลัพธ์นี้